

DRCPROP Tutorial

Carsten Schwarz, February 2012

This tutorial describes how the drcprop classes can be used to setup a simple optical systems for ray tracing.

Part 1

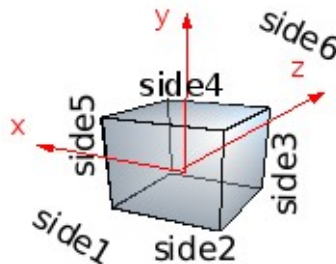
Let's start with the most simple example `test_simple_bar2.cc`

it is only a brick with the constructor

```
PndDrcOptBrick bar(17,17.5/2,100);
```

Dimensions 34x17.5x200 mm, the arguments are the half-lengths. Its center is positioned at 0,0,0. These are no physical units it could be mm,cm,m,km.

When you run doxygen to generate some class documentation, then you see there the picture of such a brick PndDrcOptBrick and the names of the surfaces.



Therefore the two surfaces side6 and side1 are the surfaces at $z=\pm 100$ and get now special functionality:

```
bar.Surface("side6")->SetReflectivity(PndDrcOptReflSilver());  
bar.Surface("side1")->SetPixel();
```

side6 is now reflecting (reflection probability as function of wavelength defined in PndDrcOptReflSilver) and side1 is a pixel, if a photon hits this side the propagation is stopped.

Now a optical system is generated which in principle could contain many volumes which are connected (future step)

```
PndDrcOptDevSys opt_system;  
opt_system.AddDevice(bar);
```

The manager holds all information about objects, one can add several optical systems. The idea was that if a detectors consists of many optical systems which are all the same, but different in position like the bars with lenses and mirrors in a barrel, they are all added to the manager

```
PndDrcOptDevManager* manager = new PndDrcOptDevManager();  
manager->AddDeviceSystem(opt_system);
```

Now a filestream is created and connected to the manager

```
fstream geo;
geo.open("Geo.C",std::ios::out);
geo<<"{"<<endl;
geo<<"    TCanvas *c1 = new TCanvas(\"c1\"); "<<endl;
if ( ((TR00T*)gROOT)->GetVersionInt() < 51600)
{
    geo<<"    TView *view = new TView(1);"<<endl;
}
else
{
    geo<<"    TView *view = TView::CreateView(1);"<<endl;
}
geo<<"    view->SetRange(-50,-50,-50,50,50,50);"<<endl;
geo<<"    Int_t i;"<<endl;
geo<<"    view->SetView(0,90,90,i);"<<endl;
// the following command sets a flag within the manager and all photons from
// now on will be traced and can be plotted by calling within root
// .x Geo.C
// .x Screen.C
//
manager->Print(geo);
//
// the intention is to play around with routines.
```

if this

```
manager->Print(geo)
```

is not commented out each photon's path element is written to a file together with the geometry of all objects. That is done in a stupid space consuming way (look to Geo.C which then is generated) that it is better for long photon lists to comment it out.

now create a particle with 0.69c 10mm under the bar and shoot in towards (0,1,1) through the bar

```
// create a list of photons in bar

XYZPoint pos(0,-10,0);
XYZVector dir(0,1,1);
double beta = 0.69;
bool photons_exist = manager->Cerenkov(pos,dir,beta); // generate photons
if (photons_exist) manager->Propagate(); // propagate photons
list<PndDrcPhoton> list_photon = manager->PhotonList(); // get list
```

if photons are generated by `manager->Cerenkov(pos,dir,beta);` then they are propagated by `manager->Propagate();` and you can access the propagated photons by accessing a list

```
for(iph=list_photon.begin(); iph != list_photon.end(); ++iph)
```

```

{
    if      ((*iph).Fate()==Drc::kPhotMeasured) icnt_measured++;
    else if ((*iph).Fate()==Drc::kPhotFlying)   icnt_flying++;
    // should never happen.
    else if ((*iph).Fate()==Drc::kPhotAbsorbed) icnt_absorbed++;
    else                                         icnt_lost++;
}

```

here it is simply the fate, but you could for example print out the coordinates, eg. (*iph).Position().X() and so on...

if you run the file you get the output

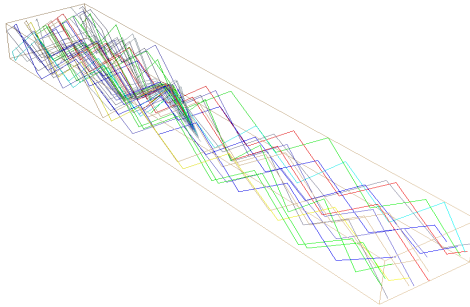
```

generated photons: 48
measured  photons: 18
absorbed  photons: 14
lost      photons: 16

```

Here, only 48 photons were generated since the speed of the particle just above the Cherenkov threshold. Absorbed in the quartz were 14 photons due to the photon attenuation length of the quartz material or due to the reflectivity of the mirror. 16 photons were lost because they have not been internally reflected. The remainin 18 photons hit the surface “side1”.

A file Geo.C is generated which can be viewed with root:



One sees the generated Cherenkov photons reflectin in the +z direction beeing reflected and hitting “side1”. The color of the photons represent their wavelength: red is red, blue is blue, the color code one finds in

```

//-----
int PndDrcPhoton::ColorNumber(double lambda) const
{
    // the colors were taken from wikipedia entry about colors.
    if (lambda< 0.1) return 17; // grey
    if (lambda> 740) return 50; // dark red
    else if (lambda> 625) return 2; // red
    else if (lambda> 590) return 42; // orange
    else if (lambda> 565) return 5; // yellow
    else if (lambda> 520) return 3; // green
    else if (lambda> 500) return 7; // cyan
    else if (lambda> 450) return 4; // blue
    else if (lambda> 430) return 9; // indigo
    else if (lambda> 380) return 39; // violet
    return 33; // blue grey
}

```

Part 2

In the next example we define a surface reflection probability for the quartz bars. Look to [test_simple_bar2a.cc](#). Including

```
#include "PndDrcOptReflGray.h"
```

the constructor

```
PndDrcOptReflGray refl;  
refl.SetReflProb(0.99);
```

is available, the reflexivity is 99% and the outside surfaces of both bars get this quantity with eg.

```
bar1.Surface("side2")->SetReflectivity(refl);
```

It is important that all devices get names. With these names one can access the objects. Internally within the classes all objects are cloned and have different pointers. Therefore, the names are needed to find these objects.

Instead of writing

```
PndDrcOptBrick bar2(17,17.5/2,100);  
bar2.SetOptMaterial(PndDrcOptMatLithotecQ0());  
bar2.SetName("bar2");  
bar2.Surface("side1")->SetPixel();  
bar2.Surface("side2")->SetReflectivity(refl);  
bar2.Surface("side3")->SetReflectivity(refl);  
bar2.Surface("side4")->SetReflectivity(refl);  
bar2.Surface("side5")->SetReflectivity(refl);
```

the following is the same

```
PndDrcOptBrick bar2(bar1);  
bar2.SetName("bar2");  
bar2.Surface("side1")->SetPixel();  
bar2.Surface("side6")->SetReflectivity(PndDrcOptReflNone());  
// (one needs the definition #include PndDrcOptReflNone.h)
```

Now we shift the bars in the right position that “side1” of “bar1” faces “side6” of “bar2”.

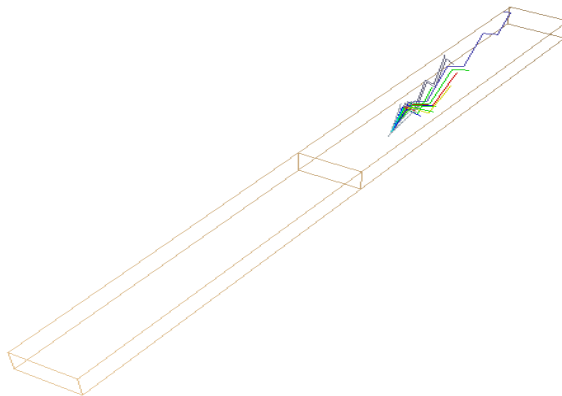
```
bar1.AddTransform(Transform3D(XYZVector(0,0, 100))); // shift by half length  
bar2.AddTransform(Transform3D(XYZVector(0,0, -100)));
```

and after adding the to the optical system, we connect the corresponding surfaces (again using names):

```
// couple surface 1 of device 1 with surface 2 of device 2  
//                               dev1   dev2   surf1   surf2  
opt_system.CoupleDevice("bar1","bar2","side1","side6");
```

One can now play with the reflection probability and see how the number of measured photons change.

For a 50% reflection probability the photons do not reach the pixel side anymore.



Part 3

The optical active volumes are not restricted to boxes, they can consist of surfaces delimited by polygons. The surfaces within these polygons need not to be flat, they can even bend like eg. Lenses. The PndDrcOptBrick is internally a combination of 6 rectangular surfaces. One can do this also by hand. We will exercise this by constructing a prism shaped expansion box. This is the example in file `test_simple_bar2b.cc`.

```
// define an expansion volume with limitin points in z-y:
//
//          p3a
//         /
//        /
//       /
//      /
//     /
//    /
//   /
//  /
// p2a
// |
// |
// |
// |
// p1a-----p4a    z <--0
//
//
// the left side will be connected to the quartz bar the right side will be the pixel.
//
//
// seen from -z it looks like
//
// p3a----p3b      Y
// |        |      ^
// |        |      |
// |        |      |
// |        |      |
// |        |      |
// |        |      |
// |        |      |
// |        |      |
// |        |      |
// p4a----p4b      x <--0
```

The coordinates of the points are given and 6 surfaces defined.

```

PndDrcSurfPolyFlat a1,a2,a3,a4,a5,a6;

a1.SetName("side+z");
a1.AddPoint(p4a);
a1.AddPoint(p4b);
a1.AddPoint(p3b);
a1.AddPoint(p3a);
a1.SetPrintColor(2); // paint it in red
a1.SetPixel();       // pixel

```

The names are needed for later access of the object. It is important, that the sequence of added points goes either clock wise or counter clock wise around the surface. Otherwise:

```

a1.AddPoint(p4a);
a1.AddPoint(p3b); // swapped
a1.AddPoint(p4b); // swapped
a1.AddPoint(p3a);

```

would create a hour glass shaped surface (not wanted).

From the surfaces one constructs the volume. It is up to the programmer, that all sides are closed and the volume has no holes.

```

PndDrcOptVol ex_box;
PndDrcOptMatLithotecQ0 quartz;
ex_box.SetOptMaterial(quartz);
ex_box.AddSurface(a1);
ex_box.AddSurface(a2);
ex_box.AddSurface(a3);
ex_box.AddSurface(a4);
ex_box.AddSurface(a5);
ex_box.AddSurface(a6);
ex_box.SetName("expansion_box");

ex_box.Surface("side-z")->SetPixel();

```

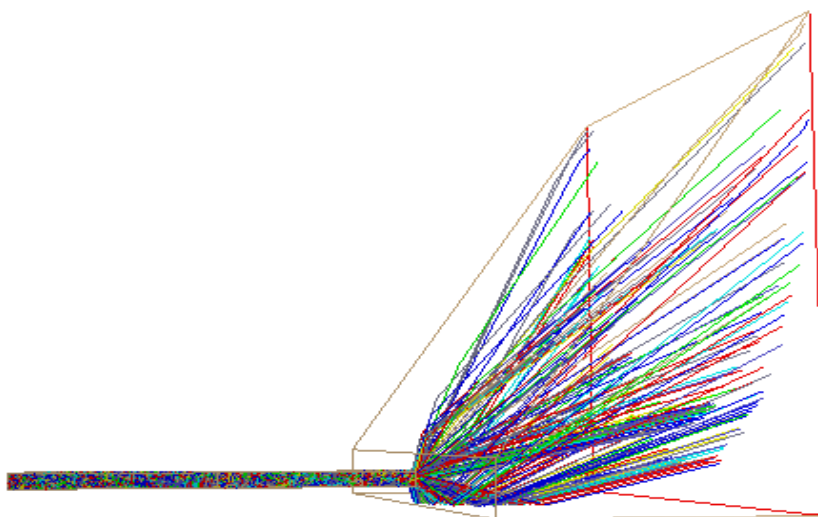
The pixel is now the last surface in -z direction. The box is added to the optical system and the connection of the surfaces are

```

// couple surface 1 of device 1 with surface 2 of device 2
//          dev1   dev2   surf1   surf2
opt_system.CoupleDevice("bar1","bar2", "side1","side6");
opt_system.CoupleDevice("bar2","expansion_box","side1","side+z");

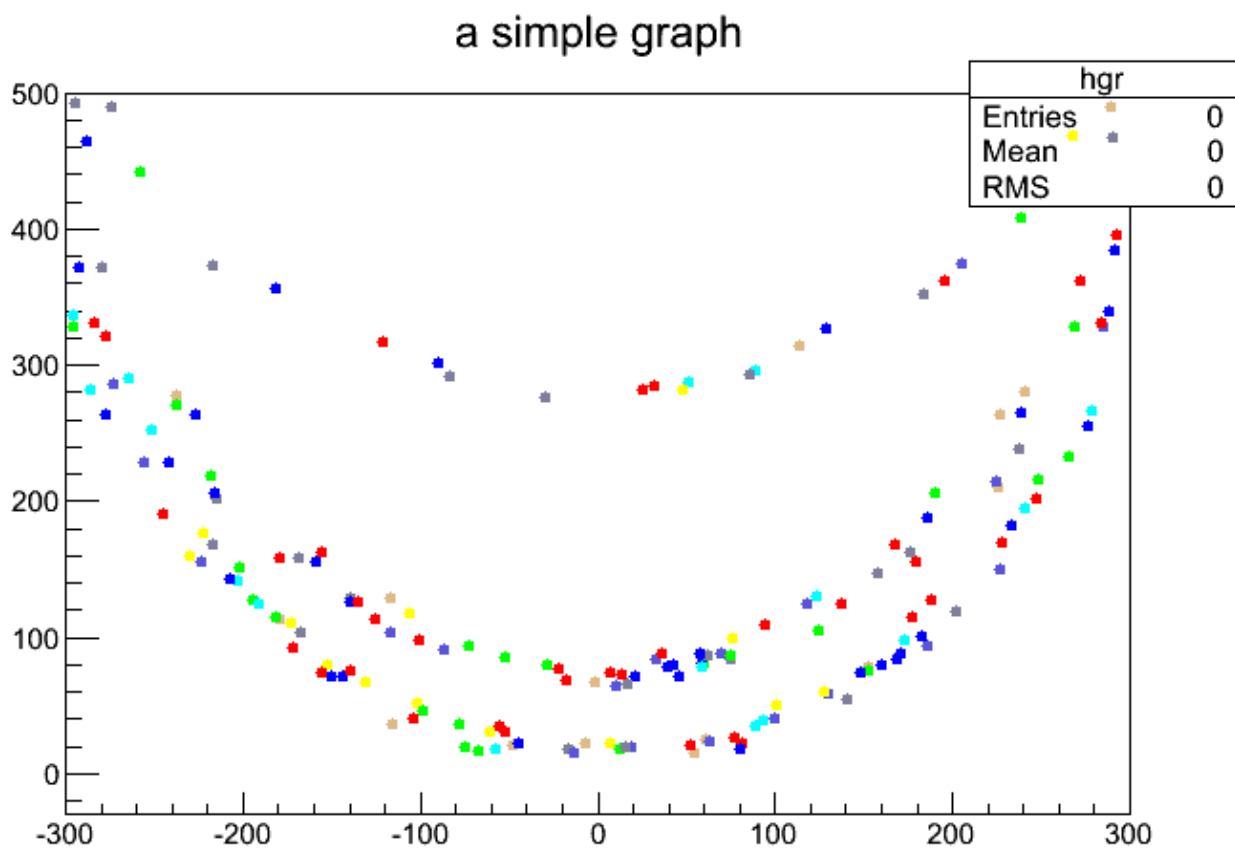
```

Running the code and “root Geo.C” shows



The example contains also a code section how to produce the coordinates of the photons which hit the backplane with colors representing the wavelength of the photons.

Run root Screen.C (Again the implementation is not very smart, but it works)



Part 4

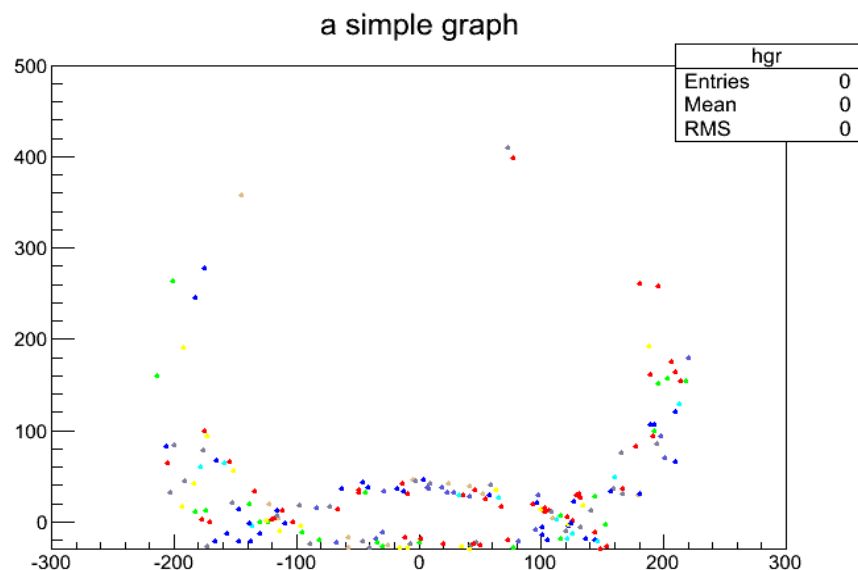
In file `test_simple_bar2c.cc` it is shown how to construct non flat surfaces. In this example a focussing mirror is attached to the downstream side of the quartz bar. For the implementation the spherical surface of a lens is taken. The lens surface gets a reflectivity.

```
PndDrcOptLens mirror(17,17.5/2,40,9999,4600);
```

```
mirror.SetOptMaterial(PndDrcOptMatLithotecQ0());
```

```
//
//
//          p16-----p26
//        /| side6 |/\
//       /|      |/\
//      /| p46-----/-p36
//     /| p1-----p2 /
//    /|      |/\
//   /|      |/\
//  /| p4-----/-p3
// /| p11-----p21 /
// | /side1      | /
// | /           | /
// /| p41-----p31
//
mirror.SetName("mirror");
mirror.Surface("side6")->SetReflectivity(PndDrcOptReflSilver()); // the mirror
mirror.AddTransform(Transform3D(XYZVector(0,0, 1000+40))); // shift by half length
+ 1000mm bar length
mirror.Surface("side21")->SetReflectivity(refl);
mirror.Surface("side26")->SetReflectivity(refl);
mirror.Surface("side31")->SetReflectivity(refl);
mirror.Surface("side36")->SetReflectivity(refl);
mirror.Surface("side41")->SetReflectivity(refl);
mirror.Surface("side46")->SetReflectivity(refl);
mirror.Surface("side51")->SetReflectivity(refl);
mirror.Surface("side56")->SetReflectivity(refl);
```

However the image does not get sharper:



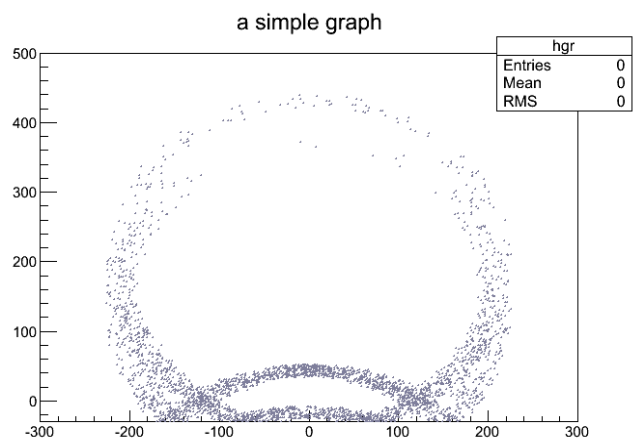
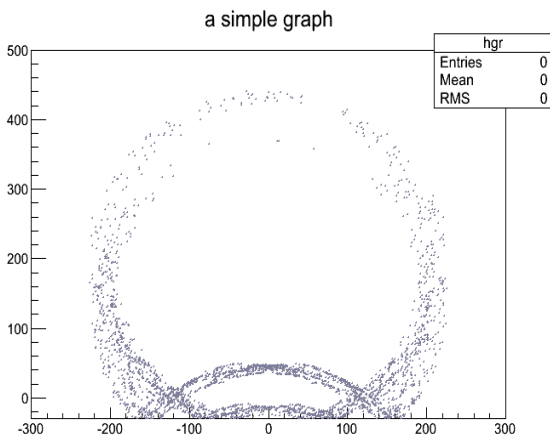
To see why, limit the wavelength range and increase the number of photons:

```
bool photons_exist = manager->Cerenkov(pos,dir,beta,10000,1e16,400,405);
```

Focussing can be switched on and off by

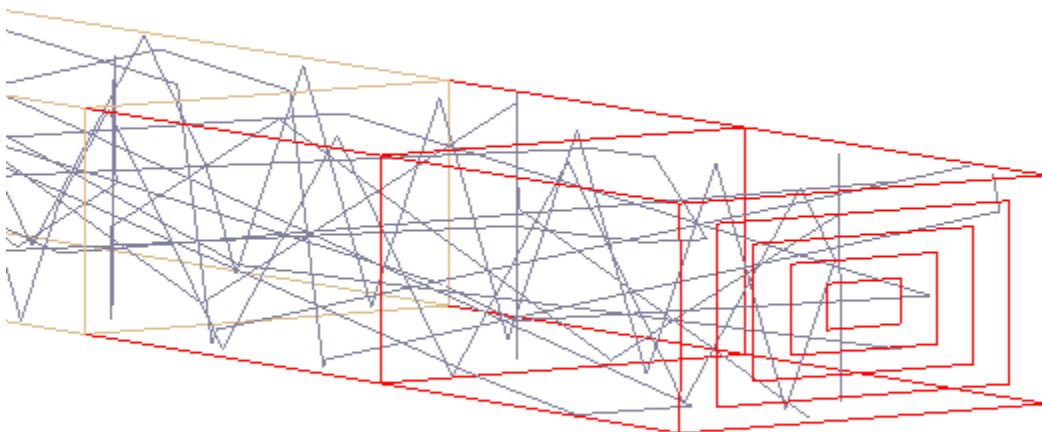
```
PndDrcOptLens mirror(17,17.5/2,40,9999,4600); // focussing  
PndDrcOptLens mirror(17,17.5/2,40,9999,9999); // non focussing
```

Now the images are (left focussed right unfocussed)



The photons which hit the top part of the screen have a longer path and are therefore unfocused.

If one uses only a few photons one can turn on the geometry for a look towards the mirror.



Photons can also manually set. If one set the option

```
OPTION opt;
```

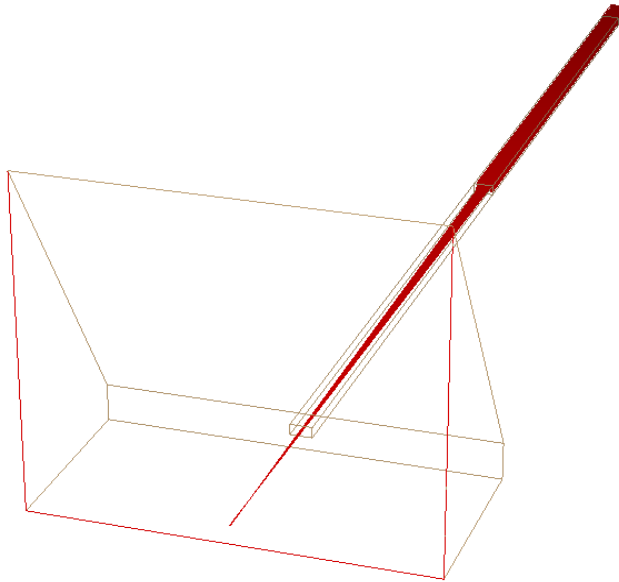
```
opt = Point;
```

the photons are set individually and are later propagated:

```
bool photons_exist = false;
list<PndDrcPhoton> list_photon;

PndDrcPhoton ph;
for (double xx=-15; xx<=+15; xx+=2)
{
    for (double yy=-7; yy<=7; yy+=2)
    {
        ph.SetPosition(XYZPoint(xx,yy,10));
        ph.SetDirection(XYZVector(0,0,+1));
        ph.SetWavelength(650);
        list_photon.push_back(ph);
    }
}
photons_exist = true;
manager->SetPhotonList(list_photon,"bar1");
if (photons_exist) manager->Propagate();           // propagate photons
list_photon = manager->PhotonList(); // get list
```

It is necessary for the propagation to tell the photons, in which volume they are. Therefore, the above parameter “bar1”. The resulting figure is:

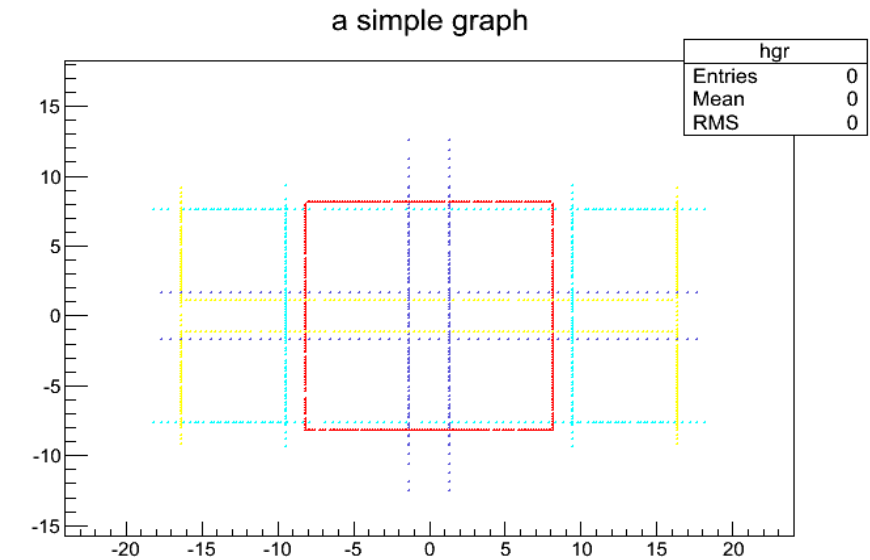


One observes photons sent downstream towards the focussing mirror and reflected back, focussed on a single point.

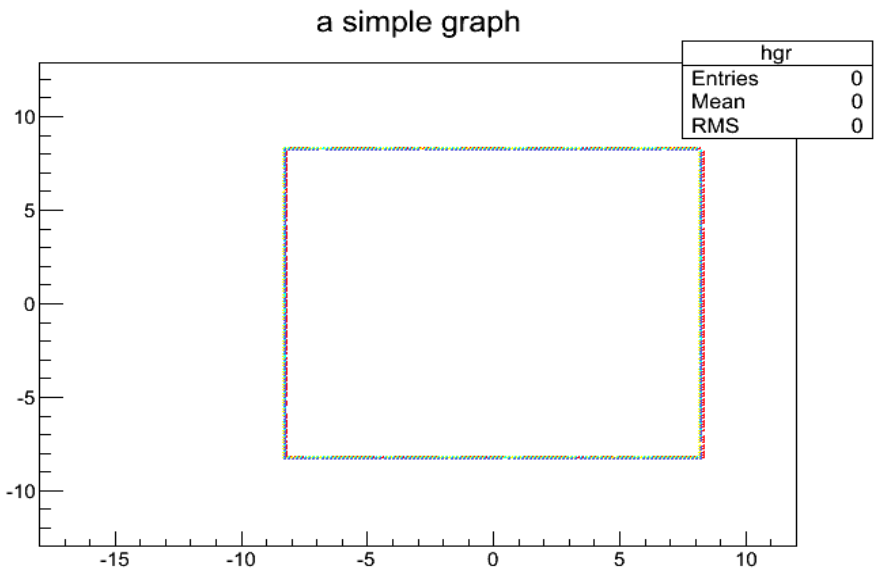
Next we create geometric figures on the screen, eg. squares, to see how the different angles are focussed. This is done by setting the option to

```
opt = Squares1;
```

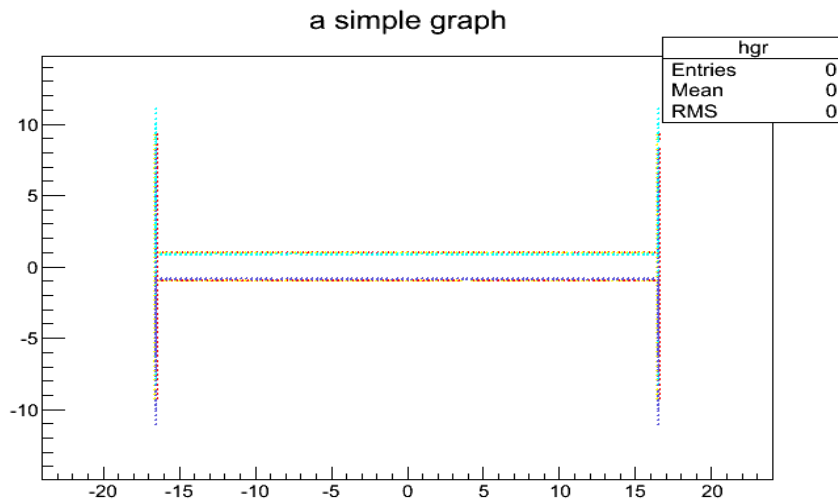
Squares with half angle 0.1,0.2,0.3, and 0.4 degrees are produced. They are red, yellow, cyan, and blue (wavelength), respectively. The red photons suffer no side reflection on their way towards the mirror and back towards the photon detector. The yellow photons suffer a reflection at the bottom and top sides, but no reflection at the left or right side.



With opt=Squares2 we test the position dependency: The colors code now if the origin for above red square is shifted by 5 mm in 4 different directions. There is a dependency, which disappears when one corrects the mirror radius from 4600mm to 4700mm in a pragmatic approach. It is a focussing effect. Now the squares are all the same:



No let's increase the emission angle to 0.2 degree, where on gets a reflection on the way towards the photon detector at the top and bottom side:



the reflection on the way from the mirror at the top and bottom side leads to different positions on the detector plane for photons emitted 5mm lower or higher. The reason is that both structures suffered different numbers of reflections. A more detailed analysis shows that hitting the mirror with two parallel beams under a certain angle, gives a focused image if you have none, one, two, and so on reflections. Rays with different reflection number hit different positions.

If one chooses now different colors for different numbers of reflections with

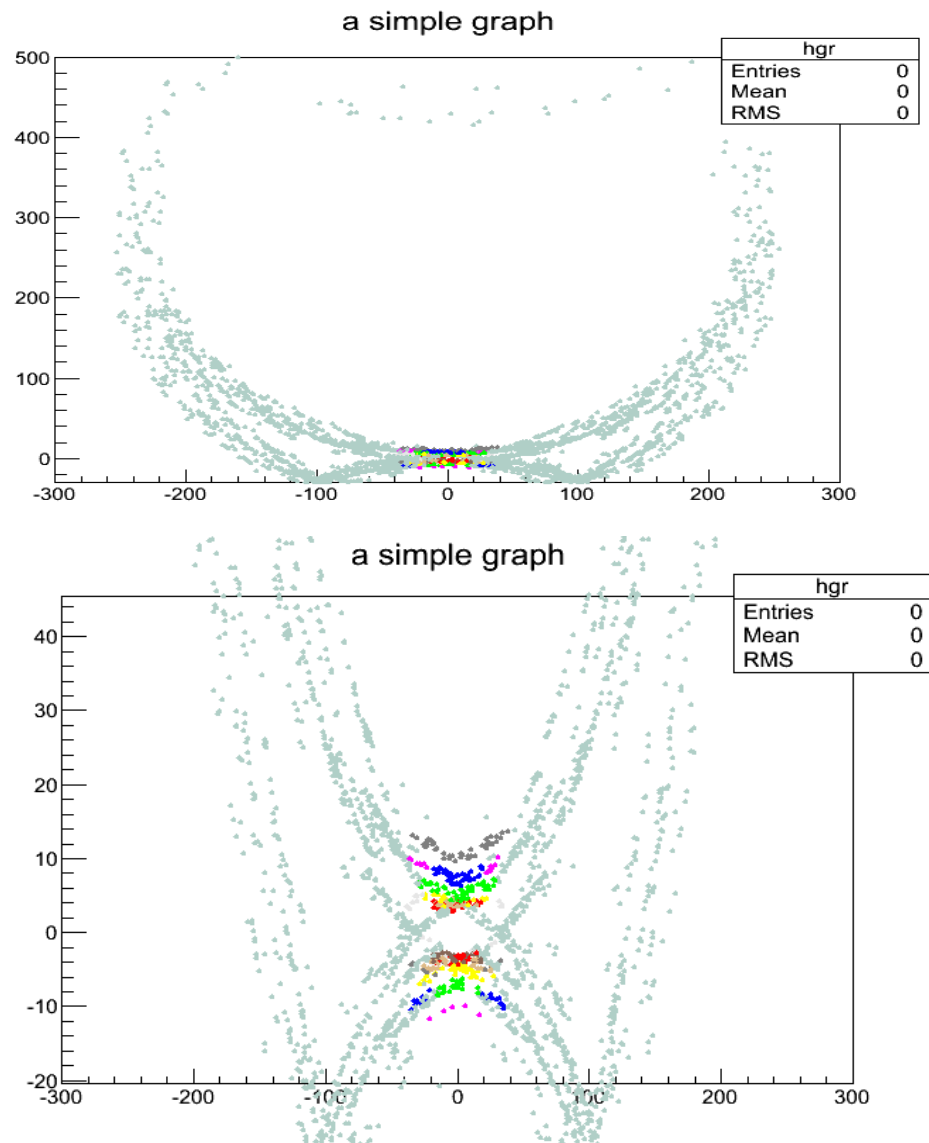
```
int irefl = (*iph).Reflections()-1; // -1 for mirror
cout<<irefl<<endl;

int icol=29;
if (irefl == 0) icol = 1; // black
if (irefl == 1) icol = 28; // brown
if (irefl == 2) icol = 2; // red
if (irefl == 3) icol = 42; // orange
if (irefl == 4) icol = 5; // yellow
if (irefl == 5) icol = 3; // green
if (irefl == 6) icol = 4; // blue
if (irefl == 7) icol = 6; // violett
if (irefl == 8) icol = 14; // gray
if (irefl == 9) icol = 18; // white

if (irefl>max) max=irefl;
if (irefl<min) min=irefl;

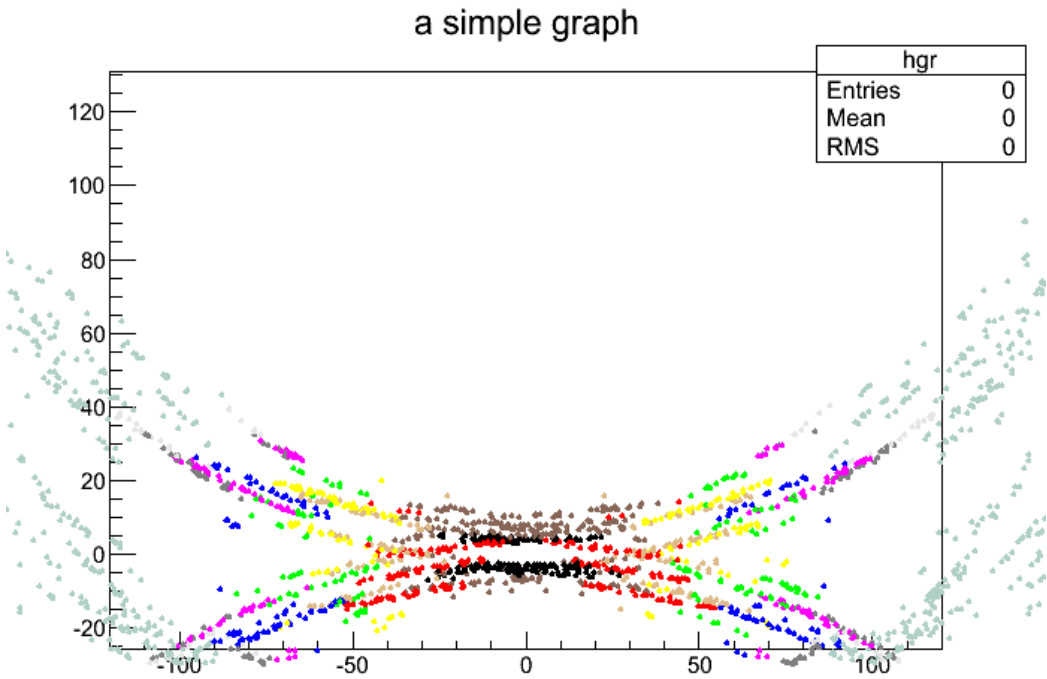
if (icol!=0)
{
    scr<<"  TMarker* t = new TMarker("<<rx<<","<<ry<<",<<20);"<<endl;
    scr<<"  t->SetMarkerColor("<<icol
    <<icol
    <<");"<<endl;
    scr<<"  t->SetMarkerSize(0.6);"<<endl;
    scr<<"  t->Draw();"<<endl;
}
}
```

one finds the following picture for a Cherenkov image:



The substructures within the Cherenkov rings originate in different number of reflections. The color code is the same as the “resistor color code”: black=0, brown=1, red=2... white=9, greenish-gray=higher. The number of reflections is without the mirror reflection.

If one looks to larger reflections the picture looks similar. Here black=0-5, brown=6-10, red=11-15 and so on:



Next, black=0-20, brown=21-40, red=41-60...

